

author1hash=847e19d487ed6b11976a9dcb3628a7f6family=International Council on
Systems Engineering, familyi=I. author1hash=b97076409949c3370d585f84edabe761family=
Automation, familyi=R.

Proposal Template Guide

Read this page before drafting the proposal.

Template-only page. For the submission PDF, change `\showguidancetrue` to `\showguidancefalse` in `metadata.tex`. This switch removes this page and the blue writing guidance.

Why the proposal is important

- **Alignment:** defines the problem, scope, deliverables, and success criteria for the group, stakeholders (supervisors), and teaching team.
- **Feasibility:** shows that the approach, schedule, resources, budget, and risk controls support completion.
- **Traceability:** connects requirements, specifications, and stakeholder objectives to the design, verification tests, and validation.

Formatting and submission

Page setup: letter paper, 1-inch margins, 12-point sans-serif body text, and 1.5 line spacing. Tables use 12 point font and may use single spacing.

Length: Sections 1–6 have an eight-page maximum. The title page, executive summary, contents, references, and appendices are excluded.

Figures and tables: number, caption, cite, and discuss each figure and table. Use appropriate units and significant digits. Choose relevant diagrams appropriately and ensure they convey your message with enough detail, using context provided in the body of the report.

Final check: remove guidance; replace bracketed fields, placeholders, and the sample logo; update citations; confirm the page count; spell-check; and submit on onQ.

Systems engineering in this proposal

The following definitions from the **INCORE!** (**INCORE!**) *Systems Engineering Handbook*, published by the **INCORE!**, convey the systems engineering (SE) perspective used in this proposal.

Systems engineering.

SE is an interdisciplinary approach and means to enable the realization of successful systems. It focuses on defining customer needs and required functionality early in the development cycle, documenting requirements, and then proceeding with design synthesis and system validation while considering the complete problem: operations, cost and schedule, performance, training and support, test, manufacturing, and disposal. SE considers both the business and the technical needs of all customers with the goal of providing a quality product that meets the user needs.

Systems engineering effort.

SE effort integrates multiple disciplines and specialty groups into a set of activities that proceed from concept to production and operation. SE considers both the business and the technical needs of all stakeholders with the goal of providing a quality system that meets their needs.

International Council on Systems Engineering **incose2015handbook**

How the proposal is graded

The proposal will be graded with an eye towards model-based systems engineering (**MBSE**): diagrams, figures, and tables should convey technical information. The body of the text should introduce ideas, state their purpose, assumptions, and limitations, provide reasoning for decisions, and interpret the information shown. More generally, the proposal is graded based on appropriately communicating engineering design, using the rubric on the next page.



Logo example: `figures/MSR_logo.png` is the current team logo. Set `\ProjectLogo` in `metadata.tex` to the team's image path, or leave it empty for a cover without a logo.

Dimension	Grading evidence
Problem, objectives, and scope	Stakeholder, system-context, use-case, objective, or requirements models define the need, system boundary, actors, operating environment, measurable objectives, scope, assumptions, and constraints. The executive summary and introduction explain the problem / motivation, project trajectory, deliverables, and how the proposed end product addresses the identified problem.
Functional description and traceability	Functional-decomposition, system block, interface, information-flow, signal-flow, state, activity, or sequence diagrams communicate system behaviour and structure. Consistent identifiers trace stakeholder needs → requirements → functions → components and interfaces. The text adequately explains these models.
Design, production, resources, and feasibility	Physical and logical architectures, function-to-component allocation models, trade-study tables, dependency networks, work-breakdown structures, milestone or Gantt diagrams, risk models, and resource or budget models show an implementable approach. The text justifies design choices, calculations, production methods, resources, budget, division of labour, contingencies, and the sequence of three to five credible milestones.
Verification and validation	Requirement traceability matrix, test flow charts, experimental test setup figures, timing or sequence diagrams, and / or other figures show how evidence progresses from components to subsystems and the integrated system. The text describes each prototype, states test methods, equipment or software, conditions, measurable acceptance criteria, required evidence, and why the planned tests sufficiently verify requirements and validate stakeholder needs.
Writing, formatting, and references	Every diagram has a descriptive caption, readable labels, defined symbols, units, directions, interfaces, requirement identifiers, and cited sources where applicable. There is consistency in terminology and identifiers. The title page, executive summary, contents, formatting, citations, and references are complete, clear, and professionally presented.

SMITH ENGINEERING

Queen's University

Department of Electrical and Computer Engineering



ELEC 490/498 Project Proposal

[Mining Software Repositories GitSkills Dataset Challenge]

Fall 2026 / Winter 2027

WRITING TEMPLATE

Submitted by Group [34]

Group member	NetID	Student number
[Noah Chung]	[NetID]	[Student number]
[Faiz Farook]	[NetID]	[Student number]
[James Kilpatrick]	[21jpk5]	[20357509]
[Naman Nagia]	[NetID]	[Student number]

Faculty supervisor(s): [Mariam Guizani]

Supervisor email: [mariam.guizani@queensu.ca]

Submission date: [07 October 2026]

Version: [1.0]

Template setup: replace the bracketed fields and develop each section into a written project proposal. Include the industry partner when applicable.

Executive Summary

Aim for approximately three-quarters of a page. Summarize the project, the contents of this proposal, two or three main milestones, and the estimated total cost. Write this section after drafting the body.

[Paragraph 1: Identify the problem, intended users or market, proposed system, and expected benefit. State the project goal and the end product.]

[Paragraph 2: Summarize the principal functions and performance targets, the proposed design and production approach, and two or three milestones.]

[Paragraph 3: List the prototypes. Summarize the testing approach, resources and total estimated cost in the stated currency, and readiness to begin the project. If further research and / or learning is necessary, list the topics and their applicability.]

It's not necessary to follow the structure above. This is just a recommendation.

Contents

Executive Summary	i
Acronyms	v
1 Introduction	1
1.1 MSR Challenge and Project Motivation	1
1.2 Project Objective and Research Direction	1
1.3 Scope and Deliverables	1
2 Functional Description	2
2.1 System overview	2
2.2 Interface and performance specifications	3
2.3 Risks and safety hazards	3
3 Design and Production Approach	4
3.1 Metric Development and Analysis	4
3.2 Milestones and Division of Labor	4
3.3 Further Exploration and Contingencies	5
4 Testing and Evaluation	6
5 Resource Requirements	7
5.1 Scheduling access to resources	7
5.2 Materials and other resources	7
6 Conclusion	8
7 References	9
A Supporting Schedule and Technical Material	10

Include this page if applicable:

List of Figures

1	Data flow from the GitSkills tables through the Python pipeline to the final metrics file.	2
---	--	---

Include this page if applicable:

List of Tables

1	Preliminary system specifications.	3
2	Proposed technical milestones and ownership.	5
3	Proposed software verification tests.	6
4	Resource access by milestone.	7
5	Preliminary cost estimate (CAD).	7

Acronyms

MBSE	model-based systems engineering
SE	systems engineering

1. Introduction

1.1 MSR Challenge and Project Motivation

The Mining Software Repositories (MSR) Mining Challenge invites researchers to investigate development practices using shared software repository datasets. The 2027 challenge provides SpecMine, containing natural-language specifications for agent-assisted development, and GitSkills, containing instructions and supporting material for AI agents. Suggested research questions provide starting points for participants' own investigations **msr2027MiningChallenge**. Our team will use GitSkills to study how agent instructions are written, reused, and maintained in software environments. These findings could help developers and researchers identify patterns in the development and evolution of this emerging software artifact.

1.2 Project Objective and Research Direction

The project builds on MSR's question of whether established software metrics such as size, churn, age, clone coverage, and readability, have meaningful equivalents for agent skills. The team will first investigate each metric independently, then examine their distributions and relationships to guide further research and inform conclusions. Potential analyses include how instruction complexity relates to revision activity and how duplicated content is distributed across skill collections. The engineering objective is to develop a reproducible pipeline that processes and validates data, calculates metrics, and produces visualizations supporting conclusions about skill development, reuse, and maintenance.

1.3 Scope and Deliverables

The initial scope is to explore GitSkills and assess which metrics support meaningful analysis of agent instructions. Research questions and scope will be refined based on available data and preliminary results. Deliverables include documented metric definitions, reproducible analysis code, visualizations, and a report interpreting the findings. Data availability will determine which analyses can be completed, and the report will explain the limitations of the measurements and conclusions.

2. Functional Description

2.1 System overview

Since this is a dataset analysis project, the system itself is fairly simple, as shown in Figure 1. GitSkills is distributed as a single SQLite database, with a Parquet copy of each table, and holds four tables: `artifacts`, `artifact_siblings`, `repos`, and `mining_runs`. The fields we need are queried from these tables into Python, where we rely on common data-analysis libraries: `pandas` and `NumPy` for data handling, `DuckDB` and `PyArrow` for querying the Parquet files, `tiktoken` for token counts, `datasketch` for MinHash similarity, `SciPy` and `powerlaw` for statistics, and `Matplotlib` for plots. A pre-processing script first transforms it for the metric scripts and filters out files that are not genuine skills (`filename`, `location_class`, `frontmatter_valid`). The cleaned data then passes to a separate calculation script for each metric, whose methods are described in Section 3. Finally, an output transformation script joins the results into a single metrics file used for analysis, plotting, and validation. The only input from outside the dataset is a small sample of cloned GitHub repositories used to validate churn.

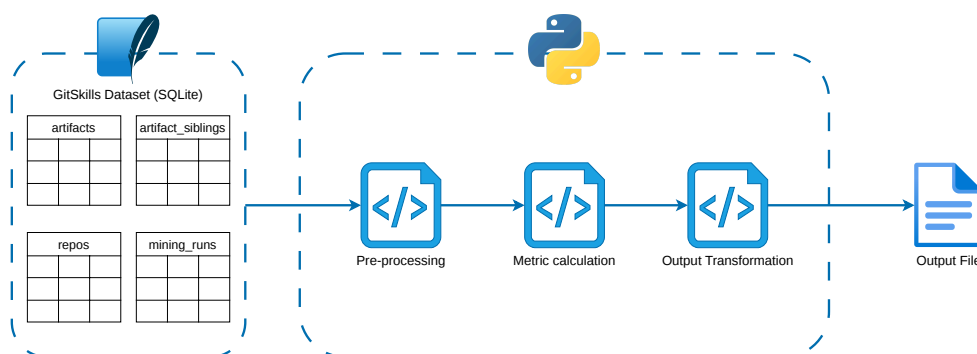


Figure 1: Data flow from the GitSkills tables through the Python pipeline to the final metrics file.

2.2 Interface and performance specifications

Table 1: Preliminary system specifications.

ID	Requirement	Target	Tolerance
IF-01	All four GitSkills tables load into Python with their documented columns	4 of 4 tables	0 missing columns
IF-02	Scripts pass data keyed by <code>file_sha</code> , with no rows lost between stages	100% of rows kept	0 rows
IF-03	Output file holds one row per distinct skill, in Parquet and CSV	1 row per skill	0 duplicates
PF-01	Full-dataset run time on one workstation	≤ 24 h	+12 h
PF-02	Peak memory use, kept low through batching	≤ 16 GB	+4 GB
PF-03	Re-running the pipeline reproduces the same metrics	Identical output	0 differences
PF-04	Precision of cross-copy lineage links on the hand-labelled sample	≥ 0.90	−0.05

2.3 Risks and safety hazards

The project is purely computational, so it carries no physical, health, or safety hazards. The main technical risk is the dataset's size (about 44 GB), which is managed by reading the per-table Parquet files in batches rather than loading everything into memory. Since the data comes from public GitHub repositories with commit authors already anonymized, we will not attempt to identify individuals, and we will respect each repository's license when quoting skill content.

3. Design and Production Approach

3.1 Metric Development and Analysis

The following outlines the proposed methods for measuring size, churn, age, clone coverage, and readability, including the data requirements, calculation procedures, and validation approach for each metric. Each metric follows four rough stages: it is prototyped on the official 13,000-skill GitSkills sample, calibrated, scaled to the full dataset, and validated.

Size. Size is classically measured in lines, but that doesn't translate well to skills. A skill mixes plain-language instructions with code blocks, and a line of prose and a line of code carry very different amounts of information; in our pilot, about 12% of a typical skill's body was code. Instead, we measure size in tokens, the units an agent actually reads text in and the units its context window is limited by. We split that count across the three parts of a skill, since an agent loads each at a different time: the *description*, read for every installed skill to decide which one to use; the *body*, loaded only once a skill is chosen; and the *bundle*, the scripts and reference files the body points to. Measuring each part separately shows not just how big a skill is, but where that size costs the agent. Lines and words are still recorded alongside tokens, so skills can be compared directly with the code bundled next to them.

Churn. Classically, churn is the number of lines added plus lines removed between commits, and it shows how much a file changes as it develops and what patterns those changes follow. Since lines don't fit skills well, churn needs adapting too, and the data adds a second complication: GitSkills records how often and when a skill was committed, but not what changed, and about two-thirds of skills in our pilot were never edited after being added. What skills tend to do instead is get copied and then modified for a new project, so we split churn into three types.

Age. [Method and validation approach.]

Clone Coverage. [Method and validation approach.]

Readability. [Method and validation approach.]

3.2 Milestones and Division of Labor

Progress will be reviewed intermittently during Friday team meetings. Each member will be required to lead at least one milestone, coordinating the work and overseeing its timely

completion. Responsibility for implementation and delivery will be shared across the team.

Table 2: Proposed technical milestones and ownership.

ID	Milestone	Due date	Lead
M1	Dataset prepared and sample records checked.	Oct. 23, 2026	Naman
M2	Initial metric calculations checked against reference examples.	Nov. 13, 2026	James
M3	Supported metrics calculated; initial tables and plots generated.	Dec. 4, 2026	Faiz
M4	Results reviewed; focused research questions documented.	Jan. 29, 2027	Noah
M5	Follow-up investigations evaluated; feasible extensions tested.	Feb. 26, 2027	Naman
M6	Final pipeline validated and results reproduced.	Mar. 12, 2027	James

3.3 Further Exploration and Contingencies

Initial findings will guide M5, which may examine instruction structure, bundled scripts and references, or reuse patterns. Additional analyses will depend on relevance, data availability, and remaining time.

Missing data or unreliable metrics will prompt revised methods or a narrower scope. Processing limits will be managed through batching or documented sampling to control time and costs. If delays threaten validation, tasks will be reassigned and optional investigations deferred. Resulting limitations will be documented.

4. Testing and Evaluation

Testing will assess the correctness and reproducibility of the analysis pipeline, from data preparation through metric calculation and visualization. Unit tests will use controlled examples with known expected results, while integration tests will check that the processing stages work together correctly. Table 3 summarizes the proposed tests and completion criteria.

Table 3: Proposed software verification tests.

Component	Proposed test	Expected evidence of success
Data preparation	Check sample records containing missing fields, duplicate content, and unusual formatting.	Records are retained, excluded, or flagged according to documented rules.
Metric calculations	Compare implemented calculations with manually calculated or independently verified examples.	Results match expected values within defined numerical tolerances.
Pipeline integration	Process a controlled sample through all analysis stages.	Record identifiers and calculated values remain consistent across intermediate and final outputs.
Reproducibility	Repeat the analysis using the same dataset, configuration, and software versions.	Tables and plotted values agree across repeated runs within defined numerical tolerances.

Evaluation will also consider whether the selected metrics and visualizations address the refined research questions. Sample outputs will be reviewed manually to confirm that labels, units, and interpretations agree with the underlying data. Any unsupported measurements or data limitations will be documented.

Testing will use the team's development environment, a unit-testing framework, and controlled samples from the selected dataset. Identified defects will be corrected and affected tests rerun before the final analysis is accepted.

5. Resource Requirements

5.1 Scheduling access to resources

Connect the technical milestones to laboratory space, equipment, fabrication, software, subscriptions, repositories, and other resources. Discuss access with the supervisor and include booking or lead-time requirements.

Table 4: Resource access by milestone.

Milestone	Resource / source	Access date / lead time
M1	[Tools and design resources]	[Date; lead time]
M2–M3	[Lab; equipment; fabrication]	[Date; booking]
M4	[Test equipment and facilities]	[Date; booking]

You can also relate each resource to the verification tests, or any other aspect of your report. Try to make use of labels / IDs so that they can be referenced consistently across the report.

5.2 Materials and other resources

Estimate costs in CAD. Include quantities and the basis for estimates; identify resources supplied through existing access. Include tax, shipping, and contingency where applicable.

Table 5: Preliminary cost estimate (CAD).

Item / source	Qty.	Unit cost	Total
[Hardware / supplier]	[n]	[Cost]	[Cost]
[Software / service / access]	[n]	[Cost]	[Cost]
[Tax; shipping; contingency]			[Cost]
Estimated total			[Total]

[State the cost assumptions, access arrangements, and supervisor or industry contributions. Explain the funding and availability needed to complete the project.]

6. Conclusion

Summarize the project and the evidence established in Sections 1–5. End with the expected contribution and the team’s readiness to proceed.

[Identify the proposed system and its purpose. Recap the objectives, scope, principal specifications, design strategy, milestones, testing approach, resources, and estimated cost. Close with the expected benefit for the intended user or application.]

7. References

Replace these entries with the sources used in the proposal. Include previous project reports, papers, books, data sheets, and web resources as appropriate. Supply enough information to retrieve each source. Cite external claims, data, and adapted figures, and record permission or license details for reused material.

A. Supporting Schedule and Technical Material

Use appendices for a detailed timeline or Gantt chart, supporting calculations, data sheets, or additional diagrams. Cite each included appendix from the body.

[Insert the detailed schedule or supporting material.]